

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN HỆ ĐIỀU HÀNH THỜI GIAN THỰC
(CNKT - MÁY TÍNH)



HCMUTE

BÁO CÁO CUỐI KỲ
HỆ THỐNG KHÓA ĐIỆN TỬ

MÃ LỚP MÔN HỌC: RTOS345264_24_1_03CLC

HỌC KỲ 1 / NĂM HỌC 2024 – 2025

Sinh viên thực hiện:

Họ và tên	Mã số sinh viên
Nguyễn Anh Minh	22119103
Huỳnh Nhật Khiêm	22119088
Đình Gia Huy	22119077

Giảng viên hướng dẫn: Huỳnh Hoàng Hà

Tp. Hồ Chí Minh, tháng 12 năm 2024.

KẾ HOẠCH PHÂN CÔNG NHIỆM VỤ LÀM BÁO CÁO MÔN HỆ ĐIỀU HÀNH THỜI GIAN THỰC

1. Mã môn học: RTOS345264_24_1_03CLC

2. Tên đề tài: HỆ THỐNG KHÓA ĐIỆN TỬ

3. Bảng phân chia công việc:

STT	Công việc	Kết quả đạt được	Người thực hiện
1	Tìm hiểu tổng quan đề tài	Tìm hiểu nguyên lý hệ thống cửa điện tử, cách hoạt động của thẻ RFID, cảm biến PIR, module RTC DS1307 và ESP32.	Đinh Gia Huy Huỳnh Nhật Khiêm Nguyễn Anh Minh
2	Thiết kế phần cứng	- Lựa chọn và lắp đặt cảm biến PIR, module RTC DS1307, RFID, Servo, Led, Button. - Thiết kế sơ đồ đấu nối phần cứng.	Đinh Gia Huy Nguyễn Anh Minh
3	Lập trình phần mềm	- Lập trình điều khiển ESP32 với FreeRTOS. - Viết thuật toán cho RFID, DS1307, PIR và Servo. - Viết thuật toán hiển thị dữ liệu lên màn hình LCD.	Huỳnh Nhật Khiêm Nguyễn Anh Minh
4	Tích hợp hệ thống	- Tích hợp phần cứng và phần mềm. - Kiểm tra sự hoạt động đồng bộ của các thành phần.	Huỳnh Nhật Khiêm Đinh Gia Huy
5	Viết báo cáo và thuyết trình	Tổng hợp kết quả, viết báo cáo cuối cùng và chuẩn bị bài thuyết trình.	Huỳnh Nhật Khiêm Đinh Gia Huy Nguyễn Anh Minh

Nhận xét của giảng viên

.....

.....

.....

.....

.....

Chữ ký của giảng viên

CHƯƠNG 1: GIỚI THIỆU

1.1 Lý do chọn đề tài

Xuất phát từ nhu cầu ngày càng cao về an ninh và tiện ích trong đời sống hàng ngày. Việc sử dụng công nghệ RFID (Radio Frequency Identification) cho phép nhận dạng và xác thực người dùng một cách nhanh chóng, không tiếp xúc, từ đó giúp kiểm soát truy cập vào các khu vực an ninh cao hoặc quản lý ra vào một cách tự động, hiệu quả.

ESP32, với kết nối Wi-Fi, Bluetooth mạnh mẽ và khả năng xử lý cao, là lựa chọn lý tưởng cho hệ thống cửa tự động. Ngoài việc điều khiển mở cửa bằng RFID, ESP32 dễ dàng tích hợp với hệ thống IoT và lưu trữ dữ liệu trên đám mây, tăng cường bảo mật và hỗ trợ quản lý từ xa.

Đề tài này nhằm tạo ra một hệ thống cửa thông minh giúp người dùng có thể khác phục được các vấn đề hay gặp phải khi sử dụng khóa cơ như kẹt ổ khóa, bảo trì ổ khóa,... bởi sự tiện lợi của nó và hơn hết tính bảo mật khi nó có chức năng thông báo các lượt thẻ mở khóa thông qua Blynk.

1.2 Mục tiêu của đề tài

- **Xây dựng hệ thống khóa cửa tự động:** Sử dụng thẻ RFID và module ESP32 để nhận diện và mở cửa một cách nhanh chóng.

- **Hệ thống cung cấp hai chế độ hoạt động theo thời gian:**

- + *Ban ngày:* Sử dụng cảm biến chuyển động PIR để tự động mở cửa khi phát hiện có người, tạo sự tiện lợi trong môi trường nhiều người qua lại như văn phòng hay cửa hàng,...

- + *Ban đêm:* Yêu cầu xác thực bằng thẻ RFID để mở cửa, tăng cường bảo mật trong thời gian ít người sử dụng.

- + *Chế độ ngày/đêm:* có thể tự động chuyển đổi dựa trên cảm biến thời gian thực hoặc có thể chuyển đổi thủ công bằng thẻ Master.

- **Tăng cường tính tự động hóa và tiện lợi:** loại bỏ việc sử dụng chìa khóa truyền thống, giúp việc mở khóa nhanh chóng và dễ dàng hơn.

- **Nâng cao kiến thức chuyên môn:** cung cấp thêm cơ hội học hỏi và ứng dụng kỹ năng lập trình ESP32 để giao tiếp với các thiết bị ngoại vi như PIR, đầu đọc RFID; hiểu

hơn về các cảm biến, kỹ thuật chuyển đổi chế độ ngày đêm và ứng dụng được vào thực tiễn nâng cao khả năng phát triển trong tương lai.

1.3 Đối tượng nghiên cứu

Đề tài hệ thống khóa cửa điện tử RFID tập trung nghiên cứu các thành phần và công nghệ liên quan để xây dựng một hệ thống khóa cửa thông minh, an toàn và tiện lợi. Ngoài ra, chế độ hoạt động ngày/đêm để đảm bảo hệ thống phù hợp với các môi trường ứng dụng thực tiễn như nhà ở, văn phòng và khu vực công cộng. Cụ thể các đối tượng nghiên cứu chính bao gồm:

- **Công nghệ RFID:** Nguyên lý hoạt động, phân loại thẻ và đầu đọc RFID, cách mã hóa và truyền nhận dữ liệu.
- **Vi điều khiển ESP32:** Khả năng kết nối Wi-Fi, Bluetooth, và lập trình để điều khiển các thiết bị ngoại vi như cảm biến PIR và đầu đọc RFID.
- **Cảm biến PIR:** Cơ chế phát hiện chuyển động, cách tích hợp với ESP32 và hiệu chỉnh để hoạt động hiệu quả trong môi trường thực tế.
- **Mô hình hoạt động của hệ thống:** Chế độ hoạt động ngày/đêm, cách phối hợp giữa RFID và cảm biến PIR để đảm bảo an toàn và tiện lợi.

1.4 Phạm vi nghiên cứu

- **Quản lý quyền truy cập:** Sử dụng ESP32 để kết nối với đầu đọc RFID và xử lý tín hiệu từ thẻ RFID để xác thực người dùng.
- **Phần cứng:** Nghiên cứu các thành phần như vi điều khiển ESP32, đầu đọc RFID, cảm biến PIR, module thời gian thực DS1307, servo và cơ chế khóa điện tử.
- **Lập trình:** Phát triển mã nguồn trên ESP32 để xử lý các tín hiệu RFID và điều khiển cửa.
- **Quản lý sự kiện:** Ghi nhận thời gian và trạng thái của cửa, có thể hiển thị thông qua màn hình LCD và LED.
- **Giới hạn quy mô ứng dụng:** cho các môi trường như nhà ở, văn phòng, hoặc các khu vực vừa và nhỏ với số lượng người dùng trong khoảng nhất định.

1.5 Phương pháp thực hiện

- **Nghiên cứu và lựa chọn công nghệ:** Tìm hiểu các loại thẻ RFID, cảm biến PIR và vi điều khiển ESP32. Chọn các thành phần phù hợp với yêu cầu của hệ thống khóa cửa điện tử.

- **Thiết kế mạch điện và kết nối phần cứng:** Thiết kế và xây dựng mạch điện kết nối ESP32 với đầu đọc RFID và cảm biến PIR. Đảm bảo các linh kiện hoạt động ổn định và tương thích với nhau.

- **Lập trình vi điều khiển ESP32:** Lập trình ESP32 để xử lý tín hiệu từ đầu đọc RFID và cảm biến PIR, điều khiển mở cửa dựa trên chế độ ngày/đêm. Cài đặt các thuật toán xử lý xác thực thẻ RFID và phát hiện chuyển động từ cảm biến PIR.

- **Tối ưu hóa và hoàn thiện:** Dựa trên kết quả thử nghiệm, tối ưu hóa phần cứng và phần mềm để cải thiện hiệu suất và độ tin cậy của hệ thống. Đánh giá khả năng mở rộng và điều chỉnh hệ thống phù hợp với các yêu cầu ứng dụng thực tế.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Hệ thống khóa cửa điện tử RFID dựa trên công nghệ nhận dạng qua sóng vô tuyến (RFID), cho phép xác thực người dùng bằng cách sử dụng thẻ RFID chứa mã nhận diện duy nhất. Vi điều khiển ESP32 đóng vai trò trung tâm, xử lý tín hiệu từ đầu đọc RFID, cảm biến PIR và điều khiển khóa cửa. Cảm biến PIR hỗ trợ phát hiện chuyển động, giúp tự động mở cửa trong chế độ ban ngày. Hệ thống hoạt động trong môi trường mạng cục bộ, tập trung vào tính bảo mật, tiện lợi và tối ưu hóa hiệu năng. Công nghệ này thường được áp dụng rộng rãi trong quản lý truy cập tại nhà ở, văn phòng và các khu vực có quy mô nhỏ.

2.1 Mô tả đề tài

Hệ thống khóa cửa điện tử (E-Door) sử dụng vi điều khiển ESP32 để kiểm soát cửa ra vào bằng các chế độ tự động hoặc thủ công. Hệ thống tích hợp cảm biến PIR, module RFID, màn hình LCD, động cơ servo, và module RTC để cung cấp các tính năng như mở cửa tự động khi phát hiện chuyển động (Daymode), mở cửa bằng thẻ RFID hợp lệ (Nightmode) và chuyển đổi chế độ tự động dựa trên thời gian thực (AutoMode). Với

FreeRTOS, các chức năng được quản lý hiệu quả qua nhiều nhiệm vụ song song, giúp tăng cường an ninh và sự tiện lợi cho người dùng.

2.2 Các tính năng chi tiết

1. Chế độ hoạt động

- Daymode (Chế độ ban ngày):

- + Cửa tự động mở khi cảm biến PIR phát hiện chuyển động.
- + Đóng cửa tự động sau một khoảng thời gian không phát hiện chuyển động.

- Nightmode (Chế độ ban đêm):

- + Cửa chỉ mở khi quét thẻ RFID hợp lệ từ danh sách được cấp quyền.
- + Cảnh báo bằng đèn LED và còi buzzer khi quét thẻ không hợp lệ.

- AutoMode (Chế độ tự động):

- + Hệ thống tự chuyển đổi giữa Daymode và Nightmode dựa trên thời gian thực được quản lý bởi module RTC.

2. Quản lý thẻ RFID

- Xác thực người dùng bằng UID của thẻ RFID.
- Quản lý quyền truy cập với danh sách thẻ được phép (allowedUIDs).
- Chỉ cho phép thay đổi chế độ hoạt động khi quét thẻ Master hợp lệ.

3. Thông báo và hiển thị

- Hiển thị thông tin:

- + Trạng thái hiện tại của hệ thống (Daymode, Nightmode, AutoMode) trên màn hình LCD, hiển thị ở dòng đầu dòng 0.

+ Hiển thị “Master pls?” khi cần quét thẻ Master để cấp quyền thay đổi chế độ bằng nút nhấn, hiển thị ở đầu dòng 0.

- + Hiển thị thời gian thực theo giờ, phút, giây; hiển thị ở dòng 1.

- Đèn LED và còi buzzer:

- + Đèn LED xanh báo hiệu mở cửa.
- + Đèn LED đỏ và còi buzzer cảnh báo khi quét thẻ không hợp lệ.

4. Động cơ servo

- Điều khiển góc mở và đóng cửa.

- Tự động khóa cửa khi không có yêu cầu mở.

5. Chuyển đổi chế độ với nút nhấn

- Nút nhấn cho phép chọn chế độ hoạt động (Daymode, Nightmode, AutoMode).
- Yêu cầu xác thực bằng thẻ Master khi chuyển chế độ.

6. Tích hợp đa nhiệm (FreeRTOS)

- Tách biệt các chức năng như quản lý cảm biến PIR, RFID, hiển thị LCD, và chế độ AutoMode thành các nhiệm vụ riêng biệt.
- Tăng hiệu suất hoạt động và tránh xung đột giữa các chức năng.

7. Bảo mật

- Cấp quyền truy cập và chuyển đổi chế độ chỉ với thẻ Master hợp lệ.
- Tự động khóa lại sau một khoảng thời gian không phát hiện chuyển động hoặc khi hoàn thành quét thẻ RFID.

2.3 Ứng dụng của đề tài

❖ Nhà ở và căn hộ thông minh

- Tích hợp trong các hệ thống nhà thông minh để kiểm soát truy cập ra vào.
- Đảm bảo an ninh cho gia đình với chế độ tự động chuyển đổi ngày/đêm.

❖ Văn phòng và công ty

- Kiểm soát nhân viên ra vào bằng thẻ RFID cá nhân.
- Hạn chế truy cập ngoài giờ hành chính bằng chế độ Nightmode.

❖ Khách sạn và khu nghỉ dưỡng

- Sử dụng thẻ RFID như thẻ phòng cho khách.
- Tự động kích hoạt chế độ Nightmode khi không có chuyển động trong khu vực.

❖ Trường học và cơ sở giáo dục

- Quản lý việc ra vào các phòng học, phòng thí nghiệm hoặc thư viện.
- Đảm bảo chỉ những người được phép (giáo viên, sinh viên) mới có thể truy cập vào khu vực.

❖ Kho hàng và khu vực lưu trữ

- Kiểm soát ra vào để bảo vệ tài sản và hàng hóa.
- Giới hạn quyền truy cập theo giờ hoạt động bằng chế độ AutoMode.

❖ Cửa hàng và siêu thị

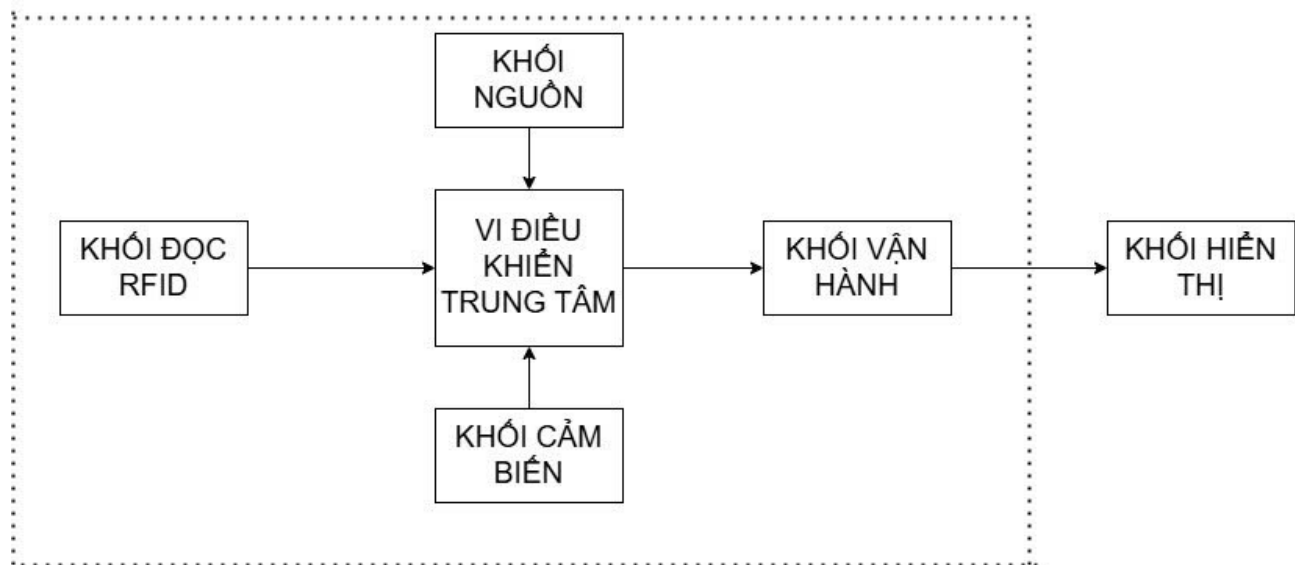
- Kiểm soát cửa ra vào tự động trong giờ mở cửa bằng cảm biến PIR.
- Chuyển sang chế độ bảo mật cao hơn (Nightmode) ngoài giờ làm việc.

❖ Khu dân cư hoặc tòa nhà chung cư

- Quản lý cửa chính của các tòa nhà với quyền truy cập bằng thẻ RFID.
- Tích hợp vào hệ thống an ninh chung để đồng bộ hóa với các camera giám sát.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 Sơ đồ khối



- **Khối nguồn:** có nhiệm vụ cung cấp nguồn điện cho các thành phần trong hệ thống và đảm bảo hệ thống hoạt động ổn định và an toàn.

- **Khối phát tín hiệu:** là thành phần quan trọng trong hệ thống, chịu trách nhiệm về việc truyền đạt thông tin và tín hiệu điều khiển từ vi điều khiển trung tâm đến các thiết bị ngoại vi. Nhiệm vụ chính của khối này là biến đổi tín hiệu điều khiển từ vi điều khiển trung tâm thành các tín hiệu thích hợp để điều khiển các thiết bị kết nối.

- **Khối cảm biến:** được thiết kế đặc biệt để nhận diện một đặc tính cụ thể của môi trường xung quanh. Nhiệm vụ chính của các cảm biến là chuyển đổi thông tin vật lý như chuyển động thành các tín hiệu điện hoặc tín hiệu khác có thể được sử dụng trong các quá trình xử lý, điều khiển, giám sát hoặc hiển thị thông tin.

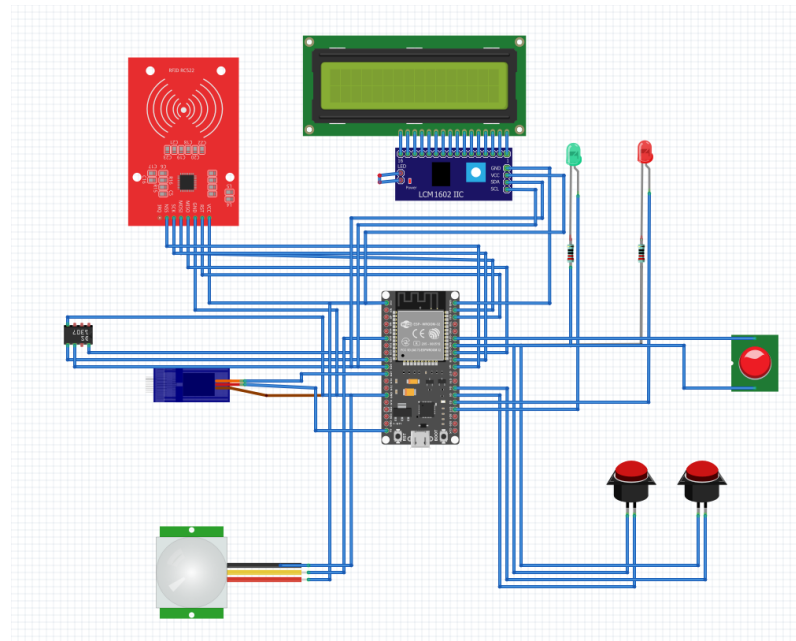
- **Vi điều khiển trung tâm:** có thể được xem như trung tâm của hệ thống nhúng, thực hiện nhiều tác vụ quan trọng để đảm bảo rằng hệ thống hoạt động đúng đắn và hiệu quả. Nó không chỉ chịu trách nhiệm cho việc xử lý dữ liệu từ cảm biến mà còn đưa ra quyết định và thực hiện các hành động cần thiết dựa trên thông tin thu thập được, đóng vai trò trung tâm quản lý thông tin và điều khiển hoạt động của hệ thống, làm cho nó trở thành một phần quan trọng của kiến trúc tổng thể.

- **Khối vận hành:** đóng vai trò quan trọng trong việc thực hiện các chức năng và nhiệm vụ hàng ngày của hệ thống. Nhiệm vụ chính của khối này là thực hiện các lệnh và quy trình được xác định trước, đồng thời đảm bảo rằng mọi hoạt động diễn ra một cách hiệu quả.

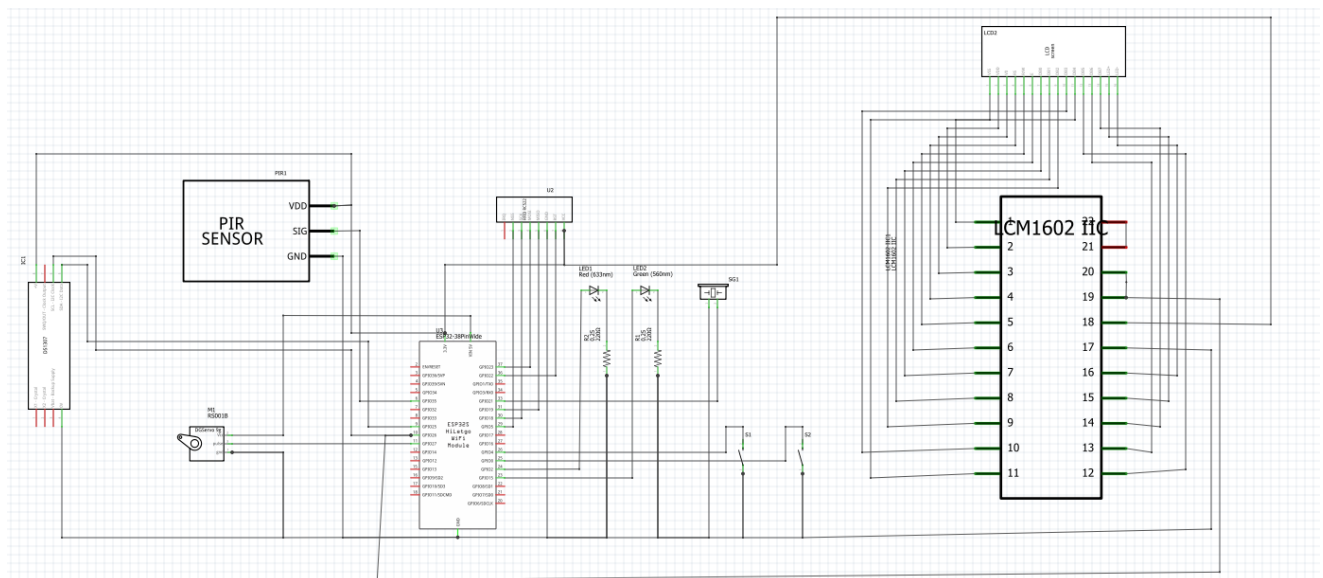
- **Khối hiển thị:** là một phần quan trọng trong cấu trúc và hoạt động của hệ thống có nhiệm vụ truyền đạt thông tin dữ liệu hoặc tín hiệu hệ thống đến môi trường bên ngoài. Nó thường là kết quả hoặc điều khiển được thực hiện bởi khối điều khiển, chẳng hạn như hiển thị ra màn hình cho phép mở cửa, quét thẻ, cảnh báo.

- **Khối đọc RFID:** Khối RFID sử dụng đầu đọc để nhận tín hiệu từ các thẻ RFID. Mỗi thẻ RFID mang một mã định danh duy nhất (UID) để nhận diện đối tượng hoặc người dùng. Sau khi đọc thông tin từ thẻ, khối RFID chuyển dữ liệu này (thường là mã UID) tới vi điều khiển trung tâm để xử lý. Dữ liệu từ thẻ được so sánh với cơ sở dữ liệu trong hệ thống để kiểm tra quyền truy cập, nhận diện đối tượng, hoặc thực hiện tác vụ đặc biệt. Nếu thông tin hợp lệ, khối RFID có thể kích hoạt các tác vụ tiếp theo.

3.2 Sơ đồ kết nối



3.3 Sơ đồ mạch chi tiết



3.4 Các linh kiện được sử dụng:

1. Cảm biến RC522 RFID



Mạch RFID NFC 13.56MHz RC522 sử dụng IC MFRC522 được sử dụng để đọc và ghi dữ liệu cho thẻ RFID tần số 13.56MHz. Chức năng chính trong hệ thống là đọc UID từ các thẻ MASTER và USER, so sánh với UID đã được cài đặt trước trong hệ thống.

Thông số kỹ thuật:

- Nguồn sử dụng: 3.3VDC
- Dòng điện: 13~26mA
- Tần số hoạt động: 13.56Mhz
- Khoảng cách hoạt động: 0~60mm (mifare1 card)
- Chuẩn giao tiếp: SPI
- Tốc độ truyền dữ liệu: tối đa 10Mbit/s
- Các loại card RFID hỗ trợ: mifare1 S50, mifare1 S70, mifare UltraLight, mifare Pro, mifare Desfire
- Kích thước: 40mm × 60mm

2. Micro servo SG90



Là động cơ có bánh răng bằng nhựa dành cho RC, có mô men lực là 1.80kg.cm tại 4.8V, được sử dụng như là chốt cửa trong hệ thống.

Thông số kỹ thuật:

- Tương thích với thư viện Arduino Servo.
- Bánh răng nhựa
- Điện áp hoạt động: 4.8 - 5VDC
- Tốc độ 4.80V(không tải): 0.12 s/60°
- Mô men lực 4.80V : 1.8 kg.cm (~0.1765 N.m)
- Góc quay: 180 độ
- Kích thước: 23.0 x12.2 x 29.0mm
- Trọng lượng: 9.0g
- Hệ thống dây điện:
- Nâu = GND
- ĐỎ = 5V
- Cam = Tín hiệu

3. Điện trở 220 Ohm



Là một linh kiện điện tử thụ động có tác dụng ngăn cản dòng điện trong mạch, dùng để mắc chung với các led trong hệ thống.

Thông số kỹ thuật:

- Model: 220R - 1W
- Sai số: 5%
- Nhiệt độ hoạt động: -55oC – 155oC
- Linh kiện xuyên lỗ: 0.5mm
- Loại: Điện trở cố định

4. Led đục xanh, đỏ 5mm



Chức năng chính trong mạch:

- Led xanh: dùng để báo hiệu khi cửa mở, quét thẻ hợp lệ.
- Led đỏ: dùng để báo hiệu khi đóng cửa, thẻ không hợp lệ.

Thông số kỹ thuật:

- Led đục
- Chiều dài chân: >20mm
- Đường kính: 5mm
- Điện áp led:
 - Đỏ: 1.8 – 2 V
 - Xanh lá: 2.8 – 3.2 V
- Dòng: 10 – 20 mA
- Độ sáng : 2000 – 4000 milicandela

5. Thẻ RFID



Thẻ nhựa RFID IC Card 13.56 MHz M1 được gán sẵn một chuỗi số UID khác nhau trên mỗi thẻ và không thể thay đổi (chỉ đọc), được sử dụng với mạch đọc RFID tần số 13.56MHz. Hệ thống sử dụng hai thẻ:

- + Thẻ User để mở khóa cửa ở Nightmode.
- + Thẻ Master hoạt động như thẻ User và để cấp quyền chuyển đổi chế độ.

Thông số kỹ thuật:

- IC: M1 Bộ nhớ: 8Kbits, 16 sectors.
- Tuổi thọ lưu trữ của vùng nhớ: 10 năm.
- Tần số giao tiếp: 13.56MHz.

- Tốc độ giao tiếp: 106kbit.
- Khoảng cách đọc: 3~5cm (tùy vào đồ đọc thẻ).
- Kích thước: $0.8 \times 85.5 \times 54\text{mm}$.
- Giao tiếp: SPI.
- Nhiệt độ hoạt động: -20°C đến 80°C

6. ESP32 NodeMCU 38 chân Type-C



ESP32 NodeMCU LuaNode32 Module đóng vai trò trung tâm điều khiển trong hệ thống, là bộ vi điều khiển trung tâm, xử lý tín hiệu từ cảm biến, điều khiển màn hình hiển thị, đèn LED và các thành phần khác trong hệ thống.

7. Buzzer



Trong hệ thống trên, buzzer (còi báo) có chức năng tạo âm thanh để báo hiệu các sự kiện hoặc trạng thái của hệ thống (Cảnh báo phát hiện chuyển động; Phản hồi trạng thái hệ thống; Báo lỗi hoặc sự cố).

Thông số kỹ thuật:

- Điện áp hoạt động: 5V DC
- Dòng điện tiêu thụ: $<25\text{mA}$
- Tần số cộng hưởng: $2300\text{Hz} \pm 500\text{Hz}$
- Cường độ âm thanh: 90dB
- Nhiệt độ hoạt động: -20°C đến $+70^{\circ}\text{C}$

8. PIR



Cảm biến PIR có chức năng chính là phát hiện chuyển động (thông qua phát hiện bức xạ hồng ngoại). Khi phát hiện chuyển động, PIR gửi tín hiệu đến ESP32 thông qua chân SIG. Tín hiệu này sẽ kích hoạt các hành động tiếp theo trong hệ thống.

Thông số kỹ thuật:

- Điện áp làm việc: 2.7~12 VDC
- Thời gian trễ: 2 giây
- Thời gian chặn: 2 giây
- Kích hoạt: có thể được lặp lại
- Phạm vi cảm biến: <100 độ, 3~5 m (yêu cầu tùy thuộc vào ống kính)
- Nhiệt độ làm việc: -20 ~ +60°C

9. Nút nhấn

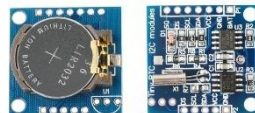


Được sử dụng làm các nút trạng thái như Automode, Nightmode, Daymode.

Nhiệt độ hoạt động: -35°C đến +85°C

- Vật liệu tiếp điểm: Niken
- Tuổi thọ: 100000 lần

10. RTC DS1307



RTC DS1307 trong hệ thống trên có các chức năng:

- Theo dõi thời gian thực
- Cung cấp thời gian cho hệ thống: DS1307 giao tiếp với ESP32 qua giao tiếp I2C, cho phép ESP32 lấy dữ liệu thời gian thực để thực hiện các chức năng liên quan:
 - Hiển thị thời gian và ngày trên màn hình LCD.
 - Kích hoạt các sự kiện dựa trên thời gian cụ thể (cảnh báo hoặc tự động bật/tắt các thiết bị vào thời điểm nhất định).

Thông số kỹ thuật:

- IC chính: RTC DS1307 + EEPROM AT24C32
- Nguồn cung cấp: 5VDC.
- Giao tiếp: I2C
- Lưu trữ và cung cấp các thông tin thời gian thực: ngày, tháng, năm, giờ, phút, giây,...
- Có pin backup duy trì thời gian trong trường hợp không cấp nguồn.
- Có ngõ ra tần số 1Hz.
- Kích thước: 27 x 28 x 8.4mm.

11. VDK ESP32 được sử dụng: ESP32 NodeMCU 38 chân Type-C

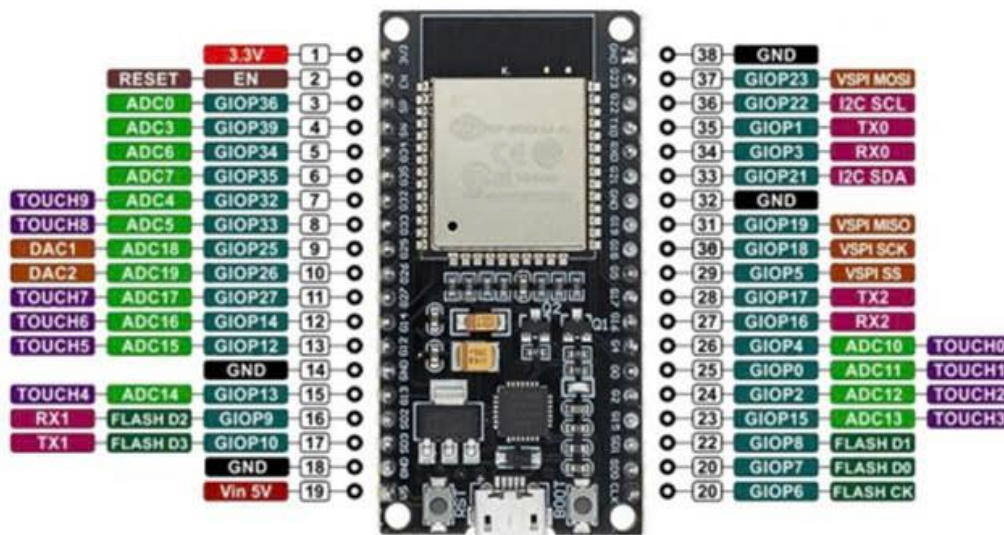


ESP32 NodeMCU LuaNode32 Module thu phát Wifi 38 chân là một module vi điều khiển mạnh mẽ với khả năng kết nối Wi-Fi và Bluetooth. Module này được tích hợp vi điều khiển ESP32 SoC, bộ nhớ flash 4MB, bộ nhớ SRAM 520KB, giao diện UART, I2C, SPI, GPIO,...

Thông số kỹ thuật:

- Điện áp hoạt động (USB): 5V DC
- Đầu vào/Đầu ra điện áp: 3.3V DC

- Công suất tiêu thụ: 5 μ A trong hệ thống treo chế độ
- Hiệu suất: Lên đến 600 DMIPS
- Tần số: lên đến 240MHz
- Bộ nhớ: 448 Kbyte ROM, 520 Kbyte SRAM, 6 Kbyte SRAM trên RTC và QSPI Hỗ trợ đèn flash / SRAM chip
- Chip USB-Serial: CP2102
- GPIO kỹ thuật số: 24 chân (một số chân chỉ làm đầu vào)
- Sơ đồ chân chi tiết:



Ý nghĩa các ký hiệu chân trên sơ đồ:

- GPIO: Thiết bị ngoại vi được sử dụng phổ biến nhất là GPIO. ESP32 có 32 chân GPIO với mỗi chân thực hiện nhiều hơn một chức năng (chỉ một chân hoạt động). Bạn có thể cấu hình chân dưới dạng GPIO hoặc ADC hoặc UART trong chương trình. Các chân ADC và DAC được xác định trước và bạn phải sử dụng các chân do nhà sản xuất chỉ định. Nhưng các chức năng khác như PWM, SPI, UART, I2C, ... có thể được gán cho bất kỳ chân GPIO nào thông qua chương trình.

- ADC: ESP32 có hai module chuyển đổi analog sang kỹ thuật số SAR 12 bit với 6 kênh và 10 kênh mỗi module. Vì vậy, khối ADC1 và ADC2 kết hợp với nhau có 1 kênh ADC 12-bit. Với độ phân giải 12-bit, các giá trị kỹ thuật số đầu ra sẽ nằm trong khoảng 0 - 4093.

- DAC: Bộ vi điều khiển ESP32 có hai kênh chuyển đổi kỹ thuật số sang analog 8 bit độc lập để chuyển đổi các giá trị kỹ thuật số sang tín hiệu điện áp analog. DAC có mạng điện trở bên trong và sử dụng nguồn điện làm điện áp tham chiếu đầu vào. Hai chân GPIO sau được liên kết với các chức năng của DAC: DAC1 - GPIO25 và DAC2 - GPIO26

- GPIO cảm ứng điện dung: SoC ESP32 có 10 GPIO cảm ứng điện dung, có thể phát hiện các biến thể về điện dung trên chân cắm do chạm hoặc tiếp cận chân GPIO bằng ngón tay hoặc bút stylus. Các GPIO cảm ứng này có thể được sử dụng để triển khai các miếng cảm ứng điện dung mà không cần bất kỳ phần cứng bổ sung nào.

- SPI: Chip Wi-Fi ESP32 có ba khối SPI (SPI, HSPI và VSPI) ở cả chế độ master và chế độ slave. SPI được sử dụng để giao tiếp với bộ nhớ Flash. Vì vậy, bạn có hai giao diện SPI.

- I2C: Có hai giao diện I2C trong ESP32 với sự linh hoạt hoàn toàn trong việc gán các chân, tức là, người dùng có thể gán các chân SCL và SDA cho cả hai giao diện I2C trong chương trình. Nếu bạn đang sử dụng Arduino IDE, thì các chân I2C mặc định là: SDA - GPIO21 và SCL - GPIO22.

CHƯƠNG 4: ỨNG DỤNG FREERTOS VÀO HỆ THỐNG

4.1 Chiến lược FreeRTOS

- **Round Robin:** Đoạn code sử dụng FreeRTOS để tạo các Task (nhiệm vụ). FreeRTOS mặc định sử dụng thuật toán lập lịch Round Robin trong các nhiệm vụ có cùng độ ưu tiên.

+ Ví dụ như các Task như TaskPIR, TaskRFID và TaskLCDDisplay đều có độ ưu tiên 1.

+ Hệ thống chia đều CPU time cho các nhiệm vụ này theo thời gian (tick) của hệ điều hành.

```
#define configUSE_TIME_SLICING 1
```

1: Time Slicing được bật.

0: Time Slicing tắt (task hiện tại chạy cho đến khi tự nhường CPU hoặc bị task ưu tiên cao hơn ngắt ngang).

- **Priority Scheduling:** Các Task trong FreeRTOS được cấp độ ưu tiên khác nhau, và hệ điều hành sẽ lập lịch theo mức độ ưu tiên này.

+ TaskModeSwitch có độ ưu tiên 2, cao hơn TaskPIR, TaskRTCModeSwitch, TaskLCDDisplay, TaskRFID với độ ưu tiên 1.

+ Điều này đảm bảo TaskModeSwitch được thực thi trước khi chuyển sang các nhiệm vụ ưu tiên thấp hơn.

```
#define configMAX_CO_ROUTINE_PRIORITIES 2
```

Cấu hình ở mức độ ưu tiên, theo như cấu hình mặc định thì sẽ có 2 mức độ ưu tiên khác nhau từ 1 đến 2. Với 1 là mức độ ưu tiên thấp nhất và 2 là mức độ ưu tiên cao nhất.

- **Chiến lược điều phối nhiều mức độ ưu tiên:** Các nhiệm vụ được nhóm lại dựa trên tính chất hoặc chức năng khác nhau, và mỗi nhóm có độ ưu tiên riêng.

+ Nhiệm vụ liên quan đến chuyển đổi chế độ (TaskModeSwitch) được ưu tiên cao nhất (2) vì ảnh hưởng đến toàn bộ hệ thống

+ Các nhiệm vụ khác như TaskPIR và TaskRFID được nhóm vào mức ưu tiên thấp hơn (1) vì chúng xử lý các sự kiện cụ thể trong chế độ đã chọn.

+ Các nhiệm vụ hiển thị thông tin (TaskLCDDisplay) có thể được thực thi với độ ưu tiên thấp hơn, nhưng vẫn phải chạy định kỳ để cập nhật màn hình.

4.2 Chế độ độc quyền hay không độc quyền

- Độc quyền: TaskModeSwitch

+ Nhiệm vụ này được gán độ ưu tiên 2, cao hơn các Task khác như TaskPIR hoặc TaskRFID với độ ưu tiên 1.

+ FreeRTOS sử dụng lập lịch ưu tiên. Khi TaskModeSwitch sẵn sàng thực thi, nó sẽ chiếm quyền CPU và ngắt các Task ưu tiên thấp hơn đang chạy.

+ Tình huống cụ thể trong code: Khi nút BUTTON_DAY, BUTTON_NIGHT, hoặc BUTTON_AUTO được nhấn, TaskModeSwitch sẽ thực thi ngay lập tức nếu không bị đình chỉ (suspend). Điều này làm gián đoạn các Task như TaskRFID hoặc TaskPIR.

- Không độc quyền: TaskPIR, TaskRFID, TaskLCDDisplay, TaskRTCModeSwitch

+ Các Task này có cùng độ ưu tiên (1), nên FreeRTOS thực hiện lập lịch theo Round Robin trong các Task này.

+ Mỗi nhiệm vụ được chia sẻ CPU theo thời gian định sẵn (tick), và chúng không giành quyền CPU từ nhau khi đang thực thi. Điều này khiến chúng không độc quyền.

+ Tình huống cụ thể trong code: Khi TaskPIR đang kiểm tra cảm biến chuyển động (digitalRead(PIR_PIN)), nếu TaskRFID cần xử lý một thẻ mới, TaskRFID sẽ phải chờ đến lượt thực thi tiếp theo mà không giành quyền CPU từ TaskPIR.

```
#define configUSE_PREEMPTION 1
```

1: Nghĩa là FreeRTOS sử dụng chế độ không độc quyền. Điều này cho phép task có ưu tiên cao hơn ngắt ngang task đang chạy.

0: Nghĩa là FreeRTOS sử dụng chế độ độc quyền. Ở chế độ này dù cho các task có ưu tiên cao hơn task đang thực thi xuất hiện thì task đang thực thi vẫn không bị ngắt bởi các task có độ ưu tiên cao hơn

4.3 Các cấp độ ưu tiên

- Có 2 cấp ưu tiên được sử dụng trong đoạn code:

+ Mức 1: được gán cho các TaskPIR, TaskRTCModeSwitch, TaskLCDDisplay, TaskRFID, các Task này thực thi theo cơ chế chia sẻ thời gian (round-robin) khi không có nhiệm vụ ưu tiên cao hơn.

+ Mức 2 (cao nhất): được gán cho TaskModeSwitch, khi Task này thực thi thì nó sẽ chiếm CPU ngay lập tức.

- Dựa vào cú pháp để xác định các cấp ưu tiên: “*xTaskCreate(TaskFunction, "Task Name", StackDepth, Parameters, Priority, Handle);*” tham số thứ 5 (Priority) là mức ưu tiên của các Task.

4.4 Mô tả chi tiết các Task trong hệ thống

```
xTaskCreate(TaskFunction, "Task Name", StackDepth, Parameters, Priority, Handle);
```

Đây là 1 cú pháp sử dụng trong freeRTOS với mục đích tạo ra các task.

Trong đó:

- TaskFunction là con trỏ đến hàm thực hiện task. Hàm này phải được định nghĩa với kiểu void và có một tham số duy nhất (void *).
- Task Name là tên của task, một chuỗi ký tự để nhận diện task.
- StackDepth là kích thước ngăn xếp (stack) được cấp phát cho task với đơn vị là words.
- Parameters là con trỏ được truyền vào task khi nó bắt đầu chạy. Tham số này có thể được dùng để cung cấp dữ liệu cụ thể cho task.
- Priority là độ ưu tiên của task. Giá trị càng cao thì ưu tiên càng lớn.
- Handle là con trỏ để lưu trữ handle của task, cho phép quản lý task sau khi nó được tạo (ví dụ: xóa hoặc tạm dừng task), giá trị của con trỏ sẽ là NULL nếu task không cần được quản lý.

⇒ Hệ thống khóa điện tử sử dụng cú pháp này để tạo ra 5 task chạy song song với nhau, thực hiện những nhiệm vụ khác nhau để điều khiển các thiết bị trong hệ thống.

1. TaskRFID:

Mô tả: Quản lý việc đọc thẻ RFID, xác thực và điều khiển đóng/mở cửa trong chế độ Night Mode (chế độ ban đêm).

Task này có độ ưu tiên là 1, với StackDepth là 4096 (word). Được quản lý bằng biến &RFIDTaskHandle.

Nhiệm vụ:

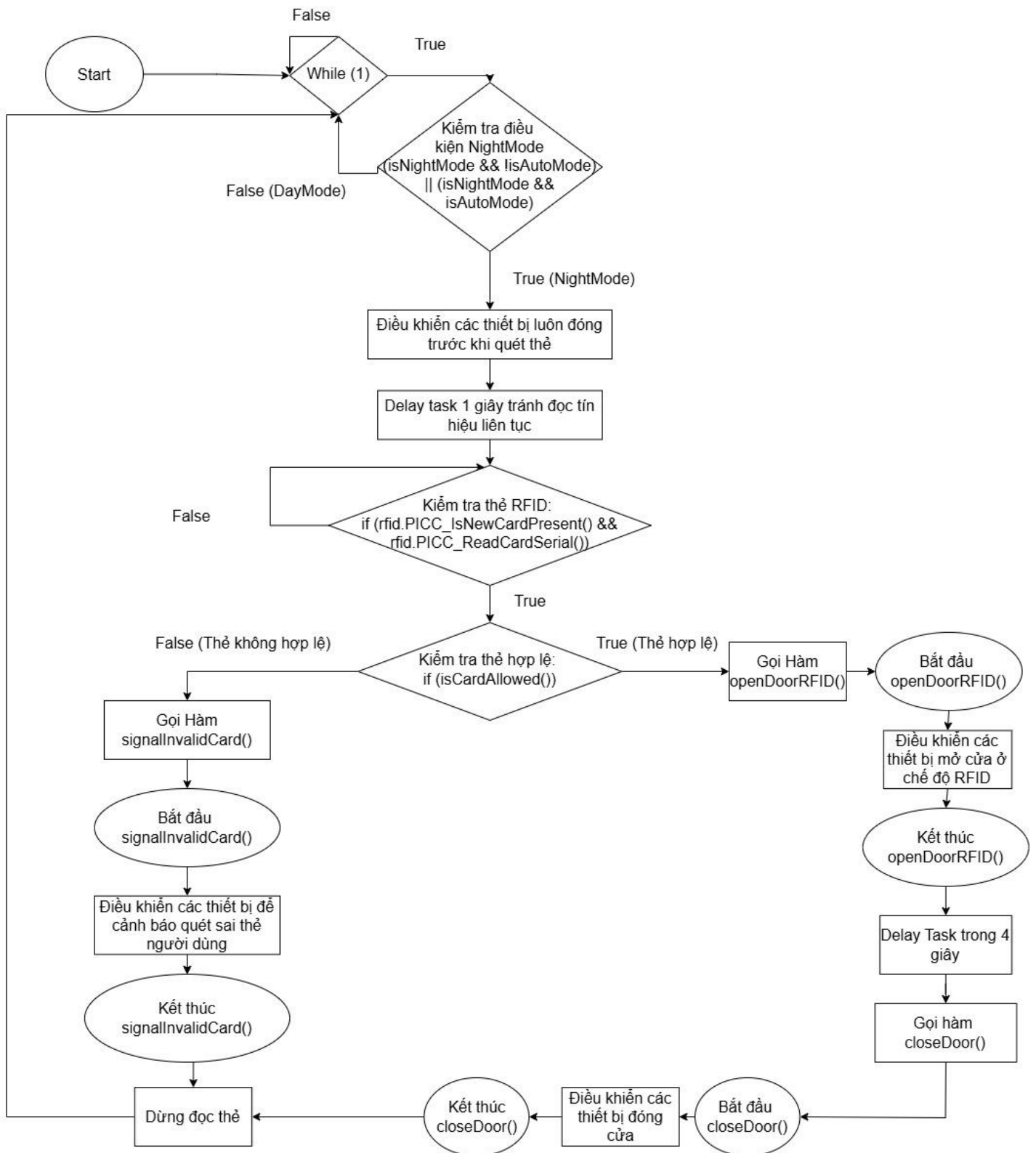
Khi hệ thống ở chế độ Night Mode (bật isNightMode), task sẽ kiểm tra xem có thẻ RFID mới được quét hay không.

Nếu có thẻ mới, task sẽ đọc thông tin thẻ và kiểm tra xem thẻ có hợp lệ (đã được đăng ký trong allowedUIDs) bằng hàm isCardAllowed().

Nếu thẻ hợp lệ, task sẽ gọi hàm openDoorRFID() để mở cửa, sau đó chờ một khoảng thời gian (RFID_OPEN_TIME) rồi đóng cửa lại bằng hàm closeDoor().

Nếu thẻ không hợp lệ, task sẽ gọi hàm signalInvalidCard() để phát tín hiệu báo động (đèn đỏ, còi).

Lưu đồ giải thuật cho TaskRFID:



2. TaskPIR:

Mô tả: Theo dõi cảm biến PIR, phát hiện chuyển động và điều khiển đóng/mở cửa trong chế độ Day Mode (chế độ ban ngày).

Task này có độ ưu tiên là 1, với StackDepth là 4096 (word). Được quản lý bằng biến &PIRTaskHandle.

Nhiệm vụ:

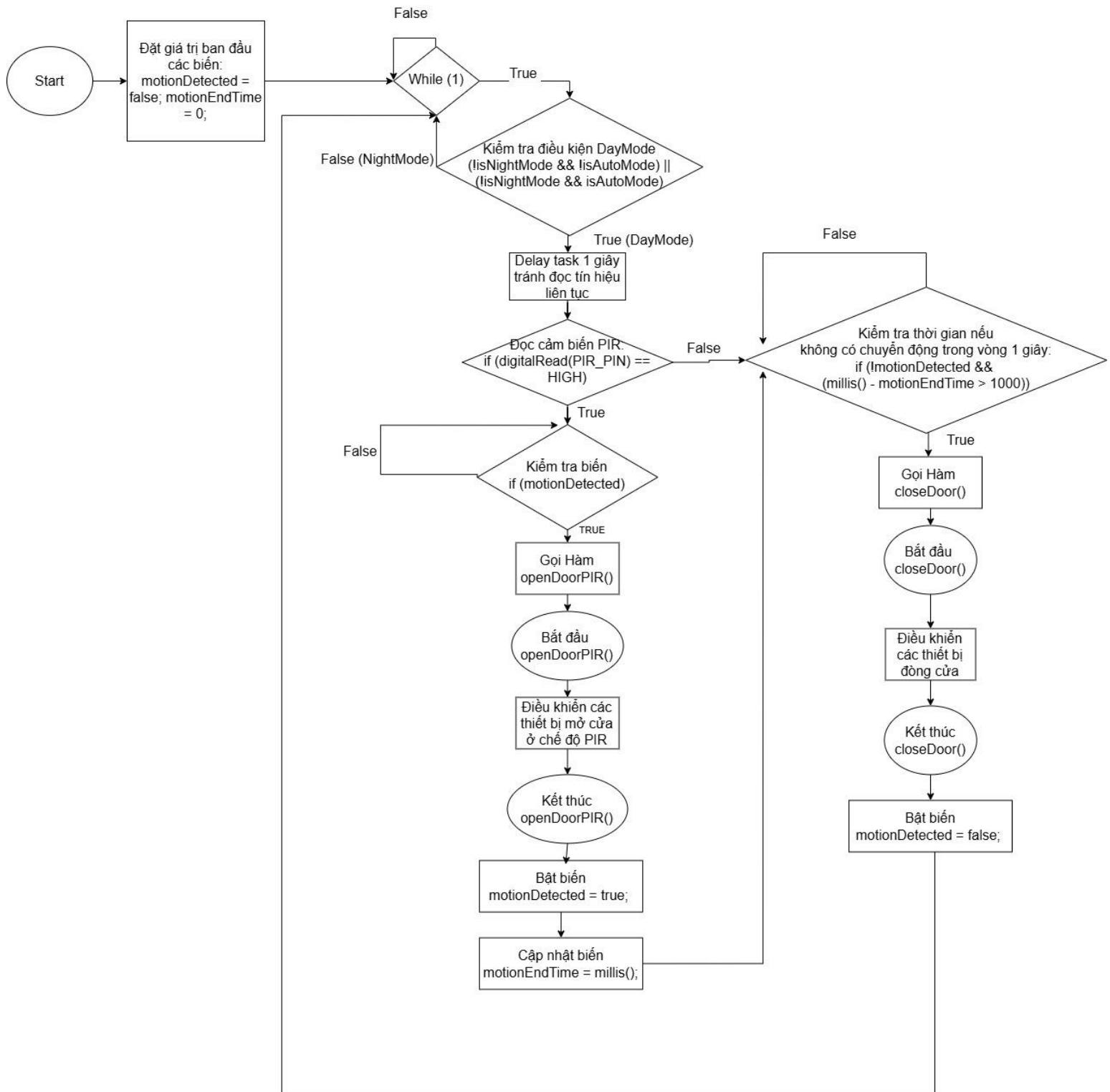
Khi hệ thống ở chế độ Day Mode (tắt isNightMode), task sẽ liên tục đọc tín hiệu từ cảm biến PIR.

Nếu phát hiện chuyển động (PIR HIGH), task sẽ kiểm tra xem cửa đã mở chưa. Nếu chưa, task sẽ gọi hàm openDoorPIR() để mở cửa.

Task cũng sẽ ghi lại thời điểm phát hiện chuyển động cuối cùng.

Nếu không phát hiện chuyển động trong một khoảng thời gian nhất định (1 giây), task sẽ gọi hàm closeDoor() để đóng cửa.

Lưu đồ giải thuật cho TaskPIR:



3. TaskRTCModeSwitch:

Mô tả: Chuyển đổi giữa chế độ Day/Night Mode dựa trên thời gian thực từ module RTC (Real Time Clock).

Task này có độ ưu tiên là 1, với StackDepth 2048 (word). Được quản lý bằng biến &RTCModeTaskHandle.

Nhiệm vụ:

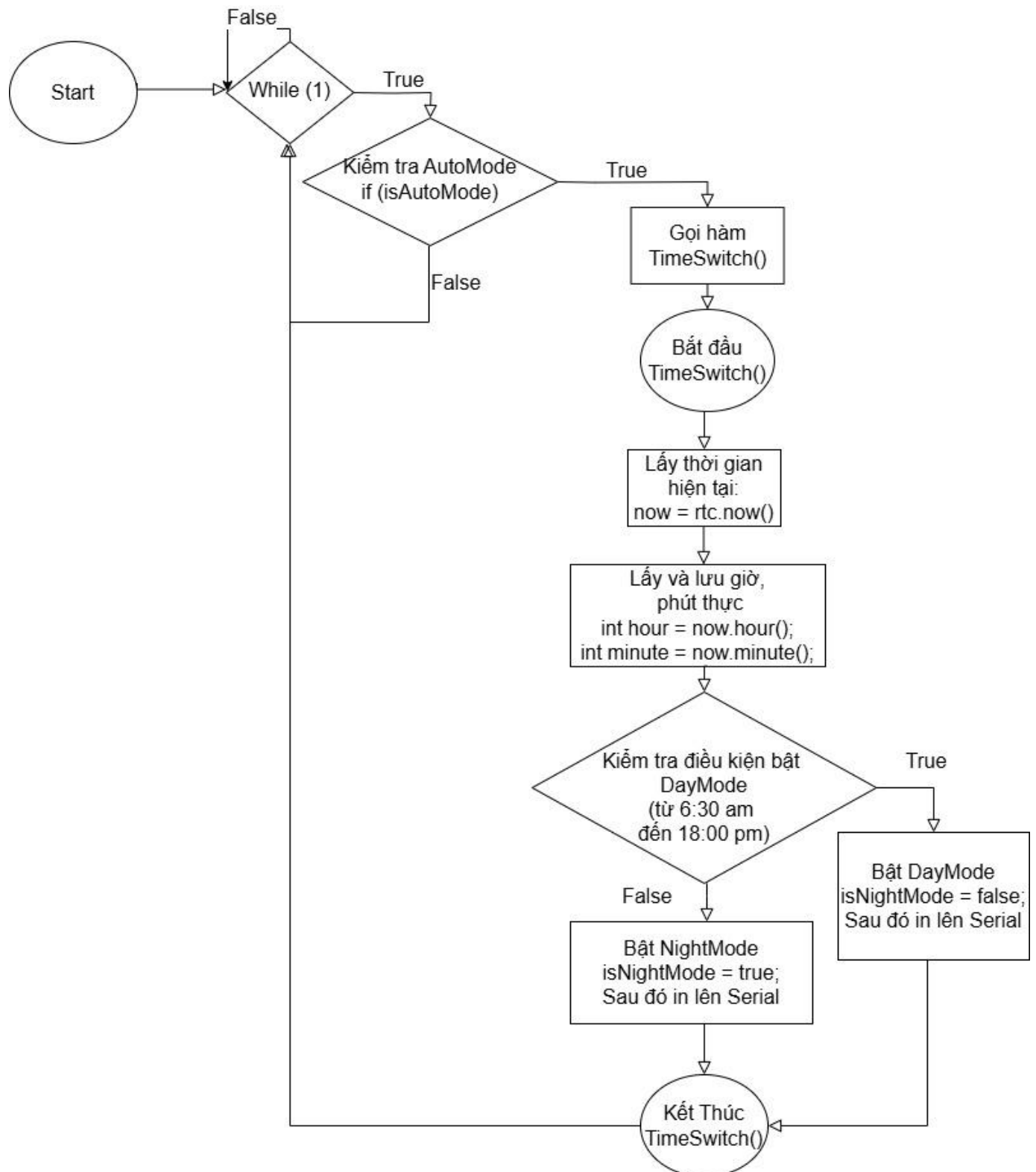
Task này chỉ hoạt động khi hệ thống ở chế độ Auto Mode (bật isAutoMode).

Task sẽ đọc thời gian hiện tại từ module RTC.

Dựa vào thời gian, task sẽ xác định và cập nhật chế độ Day/Night Mode bằng cách gán giá trị cho biến isNightMode (true cho Night Mode, false cho Day Mode).

Trong code của bạn, chế độ Day Mode được bật trong khoảng thời gian từ 6 giờ 30 phút đến 8 giờ 30 phút.

Lưu đồ giải thuật cho TaskRTCModeSwitch:



4. TaskModeSwitch:

Mô tả: Xử lý việc nhấn nút để thay đổi chế độ hoạt động của hệ thống (Day Mode, Night Mode, Auto Mode).

Task này có độ ưu tiên cao nhất trong hệ thống là 2, với StackDepth là 4096 (word).

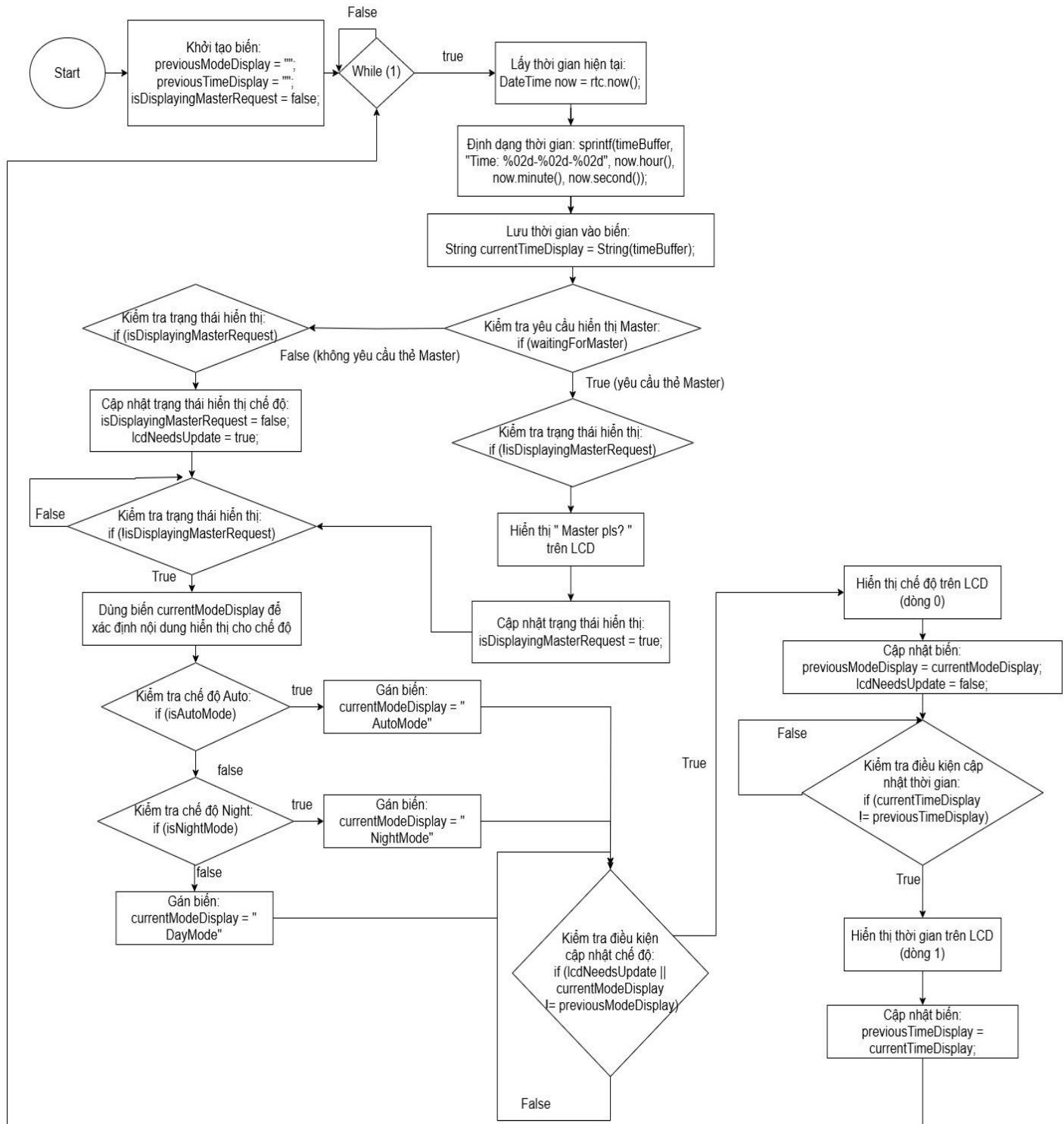
Nhiệm vụ:

Task liên tục kiểm tra trạng thái của 3 nút nhấn (Day, Night, Auto).

Khi một nút được nhấn, task sẽ yêu cầu xác thực bằng thẻ Master bằng cách gọi hàm `YeuCauMasterCard()`.

Nếu thẻ Master hợp lệ, task sẽ cập nhật chế độ hoạt động của hệ thống bằng cách thay đổi giá trị của các biến `isAutoMode` và `isNightMode`.

Lưu đồ giải thuật cho TaskModeSwitch:



5. TaskLCDDisplay:

Mô tả: Cập nhật thông tin hiển thị trên LCD, bao gồm chế độ hoạt động hiện tại và thời gian thực.

Task này có độ ưu tiên là 1, với StackDepth là 2048 (word).

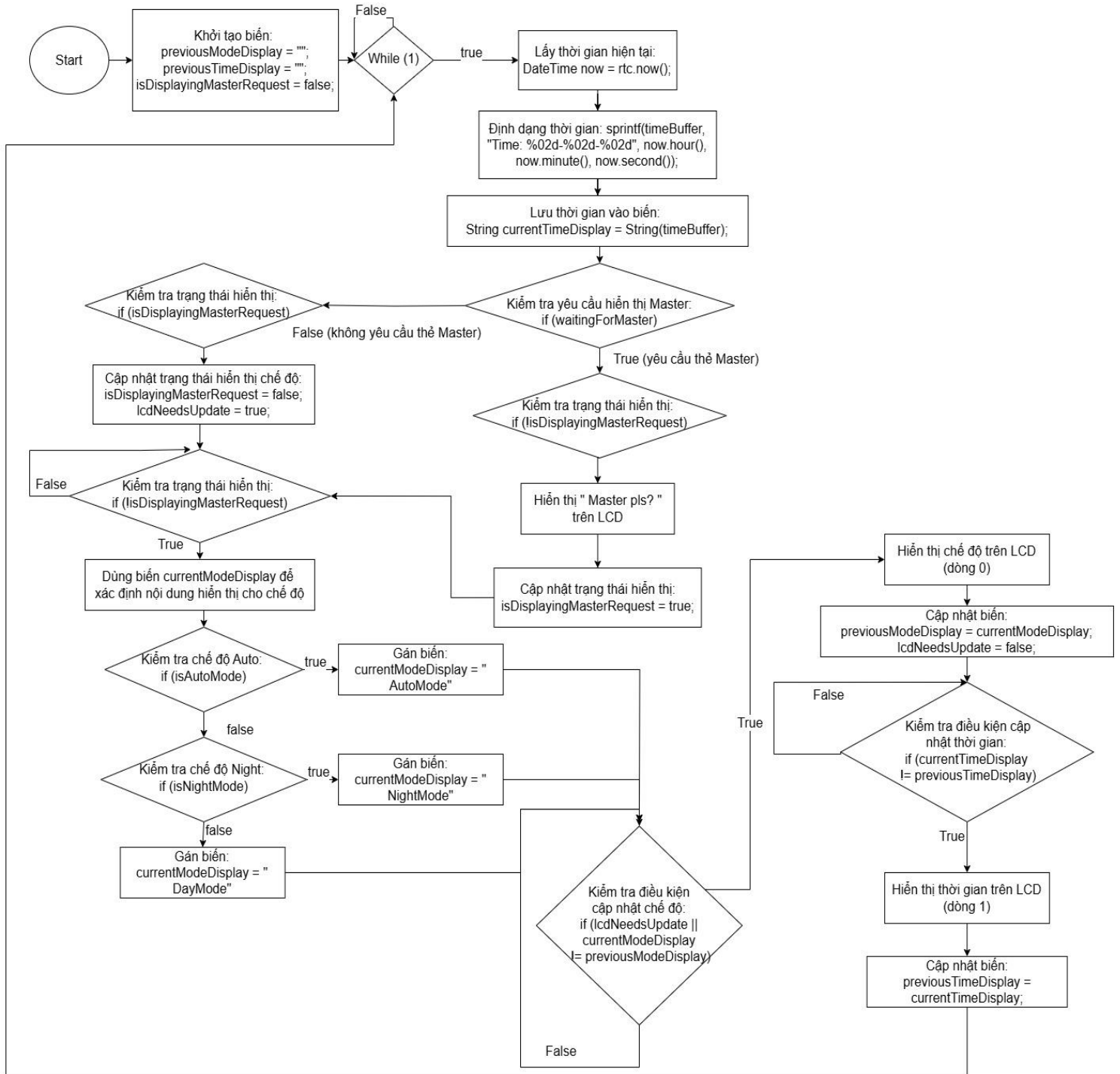
Hoạt động:

Task sẽ đọc thời gian hiện tại từ module RTC và hiển thị lên LCD dòng 1.

Task cũng sẽ hiển thị chế độ hoạt động hiện tại (Auto Mode, Day Mode, Night Mode) lên LCD dòng 0.

Khi hệ thống yêu cầu xác thực thẻ Master, task sẽ hiển thị thông báo " Master pls? " lên LCD dòng 0.

Lưu đồ giải thuật cho TaskLCDDisplay:



4.5 Đoạn code chương trình

```
// E-Door Made by Nhóm 9
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <MFRC522.h>
#include <ESP32Servo.h>
#include <RTCLib.h>
#include <FreeRTOSConfig.h>

#define SS_PIN 5
#define RST_PIN 22
#define BUZZER_PIN 21
#define RED_LED_PIN 2
#define GREEN_LED_PIN 15
#define PIR_PIN 35
#define SERVO_PIN 27
#define BUTTON_DAY 4
#define BUTTON_NIGHT 16
#define BUTTON_AUTO 17
#define SDA_PIN 25
#define SCL_PIN 26

// Khai báo LCD, MFRC522, Servo
LiquidCrystal_I2C lcd(0x27, 16, 2);
MFRC522 rfid(SS_PIN, RST_PIN);
Servo myServo;
// User UIDs với mỗi mã số thẻ có 4byte
byte allowedUIDs[][4] = {
    {0x61, 0xBC, 0x8C, 0x02}, // Thẻ user 1
    {0x03, 0x81, 0xd8, 0x0A} // Thẻ user 2
};
// MasterCard UID
byte masterUID[4] = {0x61, 0xBC, 0x8C, 0x02};

// Biến toàn cục
TaskHandle_t RFIDTaskHandle = NULL;
TaskHandle_t PIRTaskHandle = NULL;
TaskHandle_t RTCModeTaskHandle = NULL;
bool isAutoMode = true; // Biến bật Automode
bool isNightMode; // Biến bật Nightmode
// Tạo biến góc đóng/mở của Servo
const int unlockAngle = 100;
const int lockAngle = 0;
```

```

// Biến thời gian mở cửa PIR
const int RFID_OPEN_TIME = 4000;
// Biến chờ thẻ Master
bool waitingForMaster = false;
// Biến cập nhật LCD
bool lcdNeedsUpdate = true;
// Biến cập nhật cho PIR
bool motionDetected = true;
unsigned long motionEndTime = 0;
// Khai báo module DS1307
RTC_DS1307 rtc;

void setup() {
    // Khởi tạo LCD tích hợp I2C và Serial
    Serial.begin(115200);
    Wire.begin(SDA_PIN, SCL_PIN);
    lcd.init();
    lcd.backlight();
    // khởi tạo và kiểm tra khởi tạo RTC
    if (!rtc.begin()) {
        Serial.println("Không thể khởi tạo RTC!");
        while (1);
    }
    if (!rtc.isrunning()) {
        Serial.println("RTC chưa chạy, cài đặt lại giờ!");
        rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
    }
    // Khởi tạo Servo với trạng thái khóa
    myServo.attach(SERVO_PIN);
    myServo.write(lockAngle);
    // Khởi tạo RFID và các nút bấm
    SPI.begin();
    rfid.PCD_Init();
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(RED_LED_PIN, OUTPUT);
    pinMode(GREEN_LED_PIN, OUTPUT);
    pinMode(PIR_PIN, INPUT);
    pinMode(BUTTON_DAY, INPUT_PULLUP);
    pinMode(BUTTON_NIGHT, INPUT_PULLUP);
    pinMode(BUTTON_AUTO, INPUT_PULLUP);
    // Tạo các Task FreeRTOS
    xTaskCreate(TaskPIR, "PIR Task", 4096, NULL, 1, &PIRTaskHandle);
    xTaskCreate(TaskRFID, "RFID Task", 4096, NULL, 1, &RFIDTaskHandle);
    xTaskCreate(TaskModeSwitch, "Mode Switch", 4096, NULL, 2, NULL);
    xTaskCreate(TaskLCDDisplay, "LCD Display Task", 2048, NULL, 1, NULL);
}

```

```

    xTaskCreate(TaskRTCModeSwitch, "RTC Mode Switch Task", 2048, NULL, 1,
&RTCModeTaskHandle);
}
void loop() {
    // hàm loop trống
}

// Task cho AutoMode (dựa trên thời gian của RTC)
void TaskRTCModeSwitch(void *parameters) {
    while (1) {
        if (isAutoMode) {
            TimeSwitch(); // Gọi hàm TimeSwitch()
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Hàm thay đổi Day/Night theo thời gian
void TimeSwitch() {
    DateTime now = rtc.now();
    int hour = now.hour();
    int minute = now.minute();
    if (hour == 2 && minute >= 2 && minute <= 5) { // Điều kiện thời gian Daymode
        isNightMode = false;
        Serial.println("Chế độ AutoMode đang là Daymode.");
    } else { // Khoảng thời gian còn lại là của Nightmode
        isNightMode = true;
        Serial.println("Chế độ AutoMode đang là Nightmode.");
    }
}

// Task PIR cho daymode
void TaskPIR(void *parameters) {
    motionDetected = false;
    motionEndTime = 0;
    while (1) {
        if ((!isNightMode && !isAutoMode) || (!isNightMode && isAutoMode)) {
            vTaskDelay(pdMS_TO_TICKS(1000));
            if (digitalRead(PIR_PIN) == HIGH) {
                if (motionDetected) {
                    Serial.println("Phát hiện chuyển động! Mở cửa...");
                    openDoorPIR();
                    motionDetected = false;
                }
                // Cập nhật thời gian khi phát hiện chuyển động
                motionEndTime = millis();
            }
        }
    }
}

```

```

    } else if (!motionDetected && (millis() - motionEndTime > 1000)) {
        // Đóng cửa sau khi không có chuyển động trong 1 giây
        Serial.println("Không có chuyển động! Đóng cửa...");
        closeDoor();
        motionDetected = true;
    }
}
vTaskDelay(pdMS_TO_TICKS(200));
}
}

// Task cho NightMode (RFID)
void TaskRFID(void *parameters) {
    while (1) {
        if ((isNightMode && !isAutoMode) || (isNightMode && isAutoMode)) {
            myServo.write(lockAngle);
            digitalWrite(GREEN_LED_PIN, LOW);
            vTaskDelay(pdMS_TO_TICKS(1000));
            if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
                if (isCardAllowed()) {
                    openDoorRFID();
                    vTaskDelay(pdMS_TO_TICKS(RFID_OPEN_TIME));
                    closeDoor();
                } else {
                    signalInvalidCard();
                }
                rfid.PICC_HaltA();
            }
        }
        vTaskDelay(pdMS_TO_TICKS(200));
    }
}

// Biến kiểm tra thẻ hợp lệ trong danh sách
bool isCardAllowed() {
    for (int j = 0; j < sizeof(allowedUIDs) / sizeof(allowedUIDs[0]); j++) {
        bool match = true;
        for (byte i = 0; i < 4; i++) {
            if (rfid.uid.uidByte[i] != allowedUIDs[j][i]) {
                match = false;
                break;
            }
        }
    }
    if (match) {
        return true;
    }
}

```

```

    }
}
return false;
}

// Hàm này được sử dụng để cảnh báo nếu có thẻ sai bằng các thiết bị
void signalInvalidCard() {
    digitalWrite(REDA_LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
    delay(3000);
    digitalWrite(REDA_LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
}

// Hàm mở cửa của PIR
void openDoorPIR() {
    digitalWrite(GREEN_LED_PIN, HIGH);
    myServo.write(unlockAngle);
    digitalWrite(BUZZER_PIN, HIGH);
    delay(1000);
    digitalWrite(BUZZER_PIN, LOW);
}

// Hàm mở cửa của RFID
void openDoorRFID() {
    digitalWrite(BUZZER_PIN, HIGH);
    myServo.write(unlockAngle);
    for (int i = 0; i < 3; i++) {
        digitalWrite(GREEN_LED_PIN, HIGH);
        delay(200);
        digitalWrite(GREEN_LED_PIN, LOW);
        delay(200);
    }
    delay(1000);
    digitalWrite(BUZZER_PIN, LOW);
}

// Hàm đóng cửa dùng chung
void closeDoor() {
    myServo.write(lockAngle);
    digitalWrite(GREEN_LED_PIN, LOW);
    for (int i = 0; i < 3; i++) {
        digitalWrite(REDA_LED_PIN, HIGH);
        delay(200);
        digitalWrite(REDA_LED_PIN, LOW);
    }
}

```

```

        delay(200);
    }
}

// tạo biến yêu cầu quyền Master
bool YeuCauMasterCard() {
    // Tạm ngừng các task và các thiết bị
    myServo.write(lockAngle);
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
    vTaskSuspend(PIRTaskHandle);
    vTaskSuspend(RFIDTaskHandle);
    vTaskSuspend(RTCModeTaskHandle);
    waitingForMaster = true; // Kích hoạt trạng thái chờ master
    lcdNeedsUpdate = true;  // Yêu cầu cập nhật LCD
    Serial.println("Yêu cầu quét thẻ Master");
    if (waitForMasterCard()) {
        Serial.println("Cấp quyền Master thành công!");
        motionDetected = true;
        motionEndTime = 0;
        waitingForMaster = false; // Tắt trạng thái chờ master
        lcdNeedsUpdate = true;    // Yêu cầu cập nhật LCD
        vTaskResume(PIRTaskHandle);
        vTaskResume(RFIDTaskHandle);
        vTaskResume(RTCModeTaskHandle);
        return true;
    } else {
        Serial.println("Thẻ Master không hợp lệ");
        waitingForMaster = false; // Tắt trạng thái chờ master
        lcdNeedsUpdate = true;    // Yêu cầu cập nhật LCD
        delay(1500);
        // Tiếp tục các task sau khi quét thẻ Master thất bại
        vTaskResume(PIRTaskHandle);
        motionDetected = true;
        motionEndTime = 0;
        vTaskResume(RFIDTaskHandle);
        vTaskResume(RTCModeTaskHandle);
        return false;
    }
}

// Biến chờ thẻ Master, nếu quá 10s (10000 Đơn vị TG) thì sẽ timeout về lại trạng
thái hiện tại
bool waitForMasterCard() {

```

```

unsigned long startTime = millis();
const unsigned long timeout = 10000; // 10 giây
while (millis() - startTime < timeout) {
    if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
        if (isMasterCard()) {
            waitingForMaster = false; // Tắt trạng thái chờ master
            return true;
        } else {
            waitingForMaster = false; // Tắt trạng thái chờ master
            return false;
        }
    }
    vTaskDelay(pdMS_TO_TICKS(200));
}
waitingForMaster = false; // Hết thời gian chờ, tắt trạng thái chờ master
Serial.println("Hết thời gian quét thẻ Master!");
Serial.println("Quay về chế độ hiện tại");
return false;
}

// Tạo biến để kiểm tra thẻ Master
bool isMasterCard() {
    for (byte i = 0; i < 4; i++) { //so sánh từ Byte0 đến Byte3
        if (rfid.uid.uidByte[i] != masterUID[i]) {
            return false;
        }
    }
    return true;
}

// Task for switching modes with master card authentication
void TaskModeSwitch(void *parameters) {
    while (1) {
        // Kiểm tra và xử lý nút DayMode
        if (digitalRead(BUTTON_DAY) == LOW) {
            vTaskDelay(pdMS_TO_TICKS(25)); // Debounce 25ms
            if (digitalRead(BUTTON_DAY) == LOW) { // Kiểm tra lại trạng thái
                if (YeuCauMasterCard()) {
                    if (!isAutoMode && !isNightMode) {
                        Serial.println("Hệ thống đã ở chế độ Daymode thủ công.");
                    } else {
                        isNightMode = false;
                        isAutoMode = false;
                        lcdNeedsUpdate = true;
                        Serial.println("Chuyển sang chế độ Daymode thủ công.");
                    }
                }
            }
        }
    }
}

```

```

    }
}
while (digitalRead(BUTTON_DAY) == LOW) {
    vTaskDelay(pdMS_TO_TICKS(10)); // Chờ nút được nhả
}
}
// Kiểm tra và xử lý nút NightMode
if (digitalRead(BUTTON_NIGHT) == LOW) {
    vTaskDelay(pdMS_TO_TICKS(50)); // Debounce 50ms
    if (digitalRead(BUTTON_NIGHT) == LOW) { // Kiểm tra lại trạng thái
        if (YeuCauMasterCard()) {
            if (!isAutoMode && isNightMode) {
                Serial.println("Hệ thống đã ở chế độ Nightmode thủ công.");
            } else {
                isNightMode = true;
                isAutoMode = false;
                lcdNeedsUpdate = true;
                Serial.println("Chuyển sang chế độ Nightmode thủ công.");
            }
        }
        while (digitalRead(BUTTON_NIGHT) == LOW) {
            vTaskDelay(pdMS_TO_TICKS(10)); // Chờ nút được nhả
        }
    }
}
// Kiểm tra và xử lý nút AutoMode
if (digitalRead(BUTTON_AUTO) == LOW) {
    vTaskDelay(pdMS_TO_TICKS(50)); // Debounce 50ms
    if (digitalRead(BUTTON_AUTO) == LOW) { // Kiểm tra lại trạng thái
        if (YeuCauMasterCard()) {
            if (isAutoMode) {
                Serial.println("Hệ thống đã ở chế độ AutoMode.");
            } else {
                isAutoMode = true;
                lcdNeedsUpdate = true;
                TimeSwitch();
            }
        }
        while (digitalRead(BUTTON_AUTO) == LOW) {
            vTaskDelay(pdMS_TO_TICKS(10)); // Chờ nút được nhả
        }
    }
}
}

```

```

    vTaskDelay(pdMS_TO_TICKS(10)); // Giảm tải CPU
}
}

// Task for LCD display
void TaskLCDDisplay(void *parameters) {
    // Biến lưu trạng thái hiển thị trước đó để tránh cập nhật lại không cần thiết
    String previousModeDisplay = "";
    String previousTimeDisplay = "";
    bool isDisplayingMasterRequest = false;

    while (1) {
        // Lấy thời gian hiện tại từ RTC
        DateTime now = rtc.now();
        char timeBuffer[16];
        sprintf(timeBuffer, "Time: %02d-%02d-%02d", now.hour(), now.minute(),
now.second());
        String currentTimeDisplay = String(timeBuffer);
        // Kiểm tra trạng thái hiển thị Master yêu cầu
        if (waitingForMaster) {
            if (!isDisplayingMasterRequest) {
                lcd.setCursor(0, 0);
                lcd.print(" Master pls? ");
                isDisplayingMasterRequest = true; // Đánh dấu đang hiển thị yêu cầu
master
            }
        } else {
            if (isDisplayingMasterRequest) {
                isDisplayingMasterRequest = false; // Quay về hiển thị chế độ
                lcdNeedsUpdate = true; // Yêu cầu cập nhật chế độ sau khi hoàn tất quét
master
            }
        }
    }

    // Hiển thị mode nếu không phải ở trạng thái yêu cầu master
    if (!isDisplayingMasterRequest) {
        String currentModeDisplay;
        if (isAutoMode) {
            currentModeDisplay = " AutoMode :v ";
        } else if (isNightMode) {
            currentModeDisplay = " NightMode ^^ ";
        } else {
            currentModeDisplay = " DayMode :) ";
        }
        // Cập nhật dòng 0

```

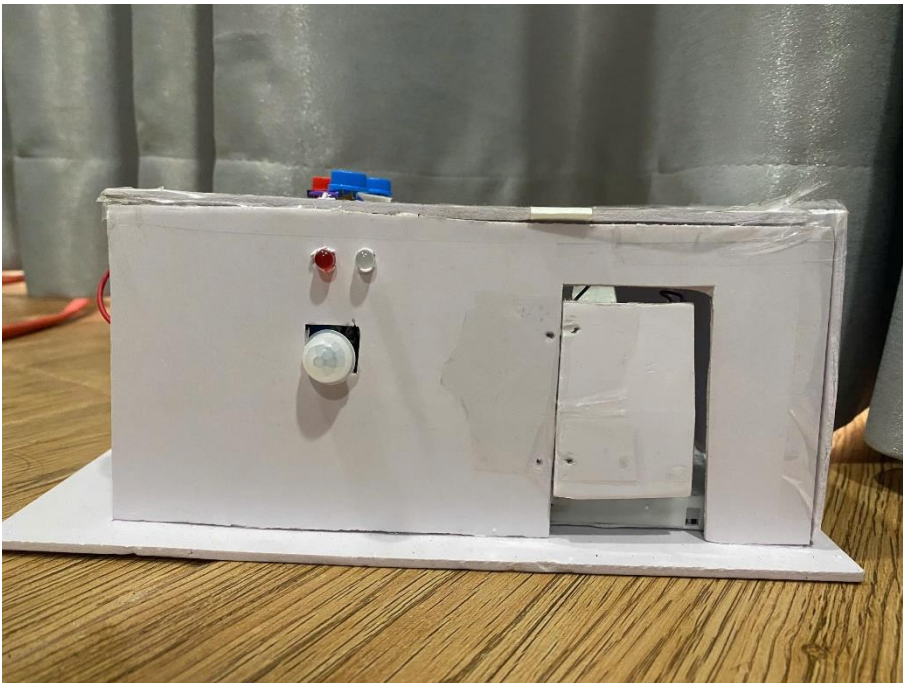
```

    if (lcdNeedsUpdate || currentModeDisplay != previousModeDisplay) {
        lcd.setCursor(0, 0);
        lcd.print(currentModeDisplay);
        previousModeDisplay = currentModeDisplay;
        lcdNeedsUpdate = false; // Reset yêu cầu cập nhật
    }
}
// Cập nhật dòng 1 (thời gian thực)
if (currentTimeDisplay != previousTimeDisplay) {
    lcd.setCursor(0, 1);
    lcd.print(currentTimeDisplay);
    previousTimeDisplay = currentTimeDisplay;
}
vTaskDelay(pdMS_TO_TICKS(500));
}
}

```

CHƯƠNG 5: THI CÔNG VÀ ĐÁNH GIÁ HỆ THỐNG

5.1 Hình ảnh thực tế sản phẩm



5.2 Demo thực tế hệ thống: <https://youtu.be/XOC3tQoj7j4>

5.3 Giải thích code: <https://youtu.be/XDFJM2z8hX0>

5.4 Đánh giá

Sau quá trình nghiên cứu, thiết kế và triển khai mạch dưới sự hướng dẫn và góp ý từ thầy Huỳnh Hoàng Hà, nhóm chúng em đã hoàn thành đề tài "Hệ thống khóa điện tử". Trong suốt quá trình thực hiện, nhóm đã gặp không ít khó khăn, từ khâu nghiên cứu lý thuyết đến thi công mạch thực tế, dẫn đến không ít sai sót. Tuy nhiên, chính những thử thách này đã giúp nhóm tích lũy kinh nghiệm quý báu và hoàn thiện sản phẩm theo đúng mục tiêu đề ra.

Hệ thống hoạt động tương đối ổn định và chính xác theo các yêu cầu đặt ra, tuy nhiên vẫn còn một số điểm cần cải thiện trong tương lai.

Nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy Huỳnh Hoàng Hà vì sự hỗ trợ tận tình và những góp ý quý giá đã giúp nhóm hoàn thành đề tài này một cách thành công.

TÀI LIỆU THAM KHẢO

- [1] ESP32 - Giới thiệu module, Thông số kỹ thuật, 08/09/2022 [Trực tuyến] <https://www.thegioiic.com/tin-tuc/esp32-gioi-thieu-module-thong-so-ky-thuat> [Truy cập 21/10/2024].
- [2] Mô-đun đọc RFID RC522 với Arduino, 06/05/2023 [Trực tuyến] <https://mecu.vn/ho-tro-ky-thuat/modun-doc-rfid-rc522-voi-arduino.w6o#%3A~%3Atext%3DRC522%20l%C3%A0%20m%E1%BB%99t%20M%C3%B4-%C4%91un%20RFID%20%C4%91a> [Truy cập 25/10/2024].
- [3] Cách sử dụng Blynk IOT với ESP32, 02/11/2024 [Trực tuyến] <https://dienthongminhesmart.com/lap-trinh-esp32/blynk-iot-vaesp32/#:~:text=C%C3%A1ch%20d%C3%B9ng%20Blynk%20v%E1%BB%9Bi%20ESP32%20l%C3%A0%20m%E1%BB%99t%20M%C3%B4-%C4%91un%20RFID%20%C4%91a> [Truy cập 19/11/2024]
- [4] Hướng dẫn ESP32 PIR, 05/10/2024 [Trực tuyến] <https://www.iotzone.vn/esp32/module-esp32/huong-dan-esp32-pir-bat-den-khico-nguoi-va-hen-gio-tatden/#:~:text=H%C6%B0%E1%BB%9Bng%20d%E1%BA%ABn%20ESP32%20PIR%20%E2%80%93%20B%E1%BA%ADt%20%C4%91%C3%A8n%20khi,quan%20s%C3%A0i%20thi%E1%BA%Bft%20b%E1%BB%8B%20qua%20Blynk%20> [Truy cập 02/11/2024]
- [5] Hướng dẫn đồng hồ thời gian thực DS1307 I2C (RTC) [Trực tuyến] [https://vn.ariat-tech.com/blog/DS1307-I2C-Real-Time-Clock\(RTC\)-Guide.html](https://vn.ariat-tech.com/blog/DS1307-I2C-Real-Time-Clock(RTC)-Guide.html)